北 京 邮 电 大 学

本科毕业设计（论文）开题报告

学院	计算机学院（国家示范性软件学院）		专业	计算机科学与技术	
学生姓名	武垚乐	学号	2022211319	班级	2022211309
指导教师姓名	张宇超	所在单位	计算机学院	职称	副教授
设计（论文）题目	（中文）面向分布式推理的零知识证明框架设计与低开销优化				
	（英文） Design and Low-Overhead Optimization of Zero-Knowledge Proof Framework for Distributed Inference				

一、 选题的背景及意义

（一） 研究背景

1. 区块链扩容与计算瓶颈
   随着 Web3.0 时代的到来，以以太坊为代表的公有区块链网络正面临着严重的性能瓶颈。为了解决 Layer 1 网络吞吐量低、交易手续费高昂的问题，Layer 2 扩容方案应运而生。在众多方案中，基于零知识证明（Zero-Knowledge Proofs, ZKP）的 zk-Rollup 方案，凭借其安全性依赖于数学原理而非博弈论假设，以及数据可用性方面的优势，被学术界和工业界公认为区块链扩容的终极方案 [1]。
   然而，zk-Rollup 的落地面临着核心挑战：证明生成（Proving）过程的计算成本极高。现代主流的通用 zk-SNARK 算法（如 Plonk）在处理复杂的智能合约逻辑时，生成的电路规模往往高达数千万甚至数亿个约束（Constraints）。生成此类证明涉及大规模的多项式运算，主要包括多标量乘法（MSM）和快速傅里叶变换（FFT）。这些运算的时间复杂度通常为超线性关系，导致单机生成证明极其缓慢 **[21]**。

2. “内存墙”与单机物理极限
   在实际工程落地中，除了计算速度慢，生成大规模电路证明还面临着严峻的“内存墙（Memory Wall）”问题。根据现有的基准测试，证明一个包含 2^28 个约束的电路，通常需要数百 GB 甚至 TB 级的内存来存储结构化参考串（SRS）和中间见证（Witness）数据 [7]。单台物理服务器不仅造价昂贵，而且受限于主板插槽物理空间和 CPU 寻址能力，很难通过无限扩展内存来解决问题。一旦物理内存不足，操作系统将通过 Swap 机制交换数据，导致计算性能呈指数级下降，严重影响了 Rollup 系统的实时性。

3. 分布式计算与 Go 语言生态的崛起
   为了突破单机物理极限，构建分布式零知识证明系统成为必然选择。通过将庞大的计算任务切分并分发至多台低成本服务器并行处理，不仅能实现计算加速，更能通过聚合多节点的内存资源来彻底解决 OOM（Out of Memory）问题 [12]。
   在技术选型上，Go 语言凭借其原生的轻量级线程（Goroutine）、高效的垃圾回收与内存管理机制、以及完善的网络编程库，已成为区块链基础设施开发的首选语言。ConsenSys 开发的 gnark 库是目前性能最强的 Go 语言 ZKP 框架 **[5]**。然而，原生的 gnark 主要针对单机多核并行进行了优化，缺乏成熟的、开箱即用的分布式集群解决方案。本课题旨在填补这一空白，基于 Go 语言生态设计并实现一个高效的分布式证明系统。

（二） 研究意义

本课题的研究具有显著的理论意义与工程应用价值：

1. 工程实践价值：构建低门槛算力网络
   本研究致力于设计一种能够在普通消费级硬件（如配置 16GB 或 32GB 内存的服务器）集群上运行大规模电路证明的系统。这将打破目前大模型推理验证和 zkEVM 对昂贵高性能服务器的依赖，为构建去中心化的证明者网络（Decentralized Prover Network）提供可行的技术范式，显著降低隐私计算技术的硬件门槛 [4][6]。

2. 学术验证价值：验证低通信复杂度架构
   传统的分布式 ZKP 方案（如 DIZK）存在严重的通信瓶颈，其网络流量随电路规模线性增长。本课题将实现并验证基于最新密码学理论（如 Pianist 架构思想）的低带宽通信方案。通过工程实验证明：利用多项式承诺的同态属性，可以将节点间通信复杂度大幅降低至常数级或对数级，从而适应非数据中心环境下的普通网络带宽 [7]。

3. 系统优化探索：Go 语言在计算密集型场景的应用
   零知识证明是典型的 CPU 和内存密集型任务。本研究将深入探索 Go 语言在处理大整数运算时的垃圾回收（GC）优化策略，以及高并发网络 I/O 下的序列化协议设计，为云原生环境下的高性能计算提供宝贵的参考经验 [8]。

二、 国内外研究现状

（一） 零知识证明算法的演进

零知识证明技术经历了从特定电路到通用电路的演进过程。Groth16 算法（2016）虽然证明极小、验证极快，但依赖于特定电路的可信设置，工程灵活性较差 **[9]**。

2019 年提出的 Plonk 算法引入了通用可更新的结构化参考串（Universal SRS），解决了可信设置复用问题。Plonk 通过置换参数（Permutation Argument）连接各个门，支持自定义门和查找表，已成为当前 zkEVM 的主流选择 **[10][21]**。虽然 2022 年提出的 HyperPlonk 试图通过多线性多项式方案消除 FFT 的瓶颈 **[11]**，但在工程成熟度上尚不如 Plonk。因此，本课题将选择 Plonk 算法作为核心协议，因其工程生态最成熟，且具有良好的代数结构，适合进行分布式改造。

（二） 分布式证明系统的挑战与突破

将 ZKP 算法分布式化并非简单的任务分发，其核心难点在于通信开销。

1. 第一代：高带宽依赖型（如 DIZK）
   2018 年，加州大学伯克利分校提出的 DIZK 系统首次在 Apache Spark 上实现了 Groth16 的分布式化。然而，DIZK 采用了朴素的分布式 FFT 算法，导致节点间需要进行全量的 All-to-All 数据混洗。实验表明，其通信复杂度为 $O(N)$，在普通的千兆网络下，巨大的通信延迟往往抵消了并行计算带来的加速收益 [12]。

2. 第二代：通信优化型（如 Pianist 架构）
   针对 DIZK 的缺陷，2024 年 IEEE S&P 会议发表的 Pianist 架构提出了革命性改进。该架构利用 KZG 多项式承诺的线性同态性，将计算任务设计为 Master-Worker 模式。Worker 节点只需在本地计算部分的承诺值（Commitment）或评估值，仅将极小的聚合结果发送给 Master。理论上，这种架构将通信量降低到了 $O(1)$ 或 $O(\log N)$ 级别 [7][13]。

（三） 当前研究空白

虽然理论层面已有 Pianist 等方案，但目前开源社区缺乏一个基于 Go 语言、开箱即用且文档完善的工程实现。现有的 gnark 库虽性能强大但仅限单机 **[14]**，而学术界的实验性代码（Research Code）往往缺乏系统级的优化（如任务调度、错误恢复）。本课题将致力于填补这一从理论到工程落地的鸿

沟。

三、 研究的基本内容和拟解决的主要问题

（一） 研究的基本内容

本课题旨在设计并实现一个名为 "Dist-Gnark" 的高性能分布式零知识证明系统。系统基于 Go 语言，复用 gnark 的底层加密原语，但在上层构建全新的分布式调度与通信架构。

1. 系统架构设计
   系统采用 Master-Slave（主从）拓扑结构，包含两类节点：

   - **Master 节点（协调者）**：负责电路管理，加载电路的元数据（不加载完整约束以节省内存）；负责任务分片（Sharding），根据接入 Worker 的数量，利用数据并行策略将 MSM 的基底向量和标量向量切分为多个子任务；负责结果聚合，接收 Worker 返回的部分承诺值，利用椭圆曲线加法进行最终聚合。

   - **Worker 节点（计算者）**：负责资源加载，仅加载分片对应的 SRS 参数，降低单节点内存压力；负责并行计算，利用本地 CPU 核心执行大规模多标量乘法（MSM）和多项式求值。

   [此处建议插入系统架构图]

2. 核心算法的分布式化封装
   本研究将重点攻克 Plonk 协议中核心算子的分布式实现：

   - **分布式 MSM**：实现基于 Pippenger 算法的分布式变体。Worker 计算局部和 $\sum s_i G_i$，Master 计算 $\sum \text{PartialSum}_j$。

   - **分布式多项式置换**：针对 Plonk 协议中的置换检查（Permutation Check），设计一种树状聚合策略。Worker 计算本地的累积乘积（Grand Product），并将边界值传递给 Master，Master 计算修正因子后回传，从而解决数据依赖性问题 **[15]**。

3. 高效通信协议设计
   为了最大化性能，本课题将摒弃传统的 JSON 或通用 Protobuf 序列化方案：

   - **自定义二进制协议**：设计紧凑的二进制帧格式，直接传输椭圆曲线点的压缩字节流（Bytes），消除冗余的类型描述字段。

   - **流式传输**：利用 Go 的 io.Reader 和 io.Writer 接口，实现边接收边反序列化，减少内存拷贝开销。

（二） 拟解决的主要问题

1. 海量数据的切分策略
   如何将 GB 级的 SRS 和电路数据高效地切分并加载到 Worker 节点？如果切分不均，会导致负载不均衡。本研究拟解决基于电路规模的自适应切分问题。

2. Go 语言的 GC 抖动
   ZKP 计算涉及数亿次的大整数对象创建，极易触发 Go 语言的垃圾回收导致 STW（Stop-The-World）。本研究拟通过引入对象池（sync.Pool）技术来复用内存对象，解决内存管理难题。

3. 网络延迟的掩盖

在分布式环境中，网络 I/O 可能阻塞计算。拟通过异步 I/O 和流水线技术（Pipelining），实现计算与传输的重叠（Overlap），最大化 CPU 利用率。

四、 研究方法及措施

（一） 研究方法

1. 文献分析法
   深入研读 Pianist、Plonk 及 Gnark 的相关论文和技术文档，掌握分布式零知识证明的数学原理[16]。

2. 原型迭代法
   采用敏捷开发模式，分阶段推进：

   - 阶段一：单机模拟分布式（利用 Go Channel 模拟网络），验证逻辑正确性。
   - 阶段二：实现基础 RPC 通信，打通节点间交互。
   - 阶段三：集成加密算法，完成完整系统构建。

3. 控制变量实验法
   在相同的硬件环境下，通过调整 Worker 节点数量（1, 2, 4, 8），对比系统的总耗时、内存峰值和网络流量，量化系统的性能指标 [17]。

（二） 具体技术措施

1. **开发技术栈**

   - **语言**：Go 1.21+（利用泛型优化代码复用）。
   - **核心库**：consensys/gnark (v0.10.0) 和 consensys/gnark-crypto **[18]**。
   - **通信**：Go 标准库 net/rpc 或基于 TCP 的自定义协议。

2. **关键实现细节**

   - **Master 端实现**：维护一个 Worker 注册表，使用心跳机制监测节点状态。实现任务调度器，将 []G1Point 切片分发给不同 Worker。
   - **Worker 端实现**：实现无状态计算服务，接收二进制数据包，调用 gnark-crypto 的 MultiExp 接口进行计算。
   - **数据压缩**：在网络层引入 Snappy 压缩算法，针对稀疏的电路数据（如选择子多项式）进行压缩传输，以 CPU 算力换取网络带宽。

五、 研究工作的步骤与进度

本毕业设计预计周期为 16 周，具体安排如下：

1. 第 1-2 周：文献调研与环境搭建
   阅读 Plonk 协议原文及 Pianist 分布式架构论文 [7][19]。搭建 Go 开发环境，安装 gnark 依赖，跑通官方示例代码，分析其底层数据结构。

2. 第 3-5 周：系统骨架开发
设计 Master-Worker 通信协议（RPC 接口定义）。实现基础的节点发现、连接管理和简单的任务分发功能（如字符串回显测试），确保基础通信链路畅通。

3. 第 6-9 周：核心算法集成（攻坚阶段）
移植 MSM 算法，将单机 MSM 拆解为分片任务，实现 Master 聚合逻辑。移植 Plonk 流程，实现置换多项式和商多项式的分布式计算逻辑。注：此阶段将参考开源实现逻辑进行工程化重构 [20]。

4. 第 10-12 周：性能优化与测试
引入 sync.Pool 优化内存。在局域网多节点环境下（或 Docker 模拟）进行基准测试，收集加速比数据。

5. 第 13-14 周：数据分析与论文撰写
整理实验图表（耗时对比图、内存占用图）。撰写毕业设计论文，详细阐述系统设计与实验结果。

6. 第 15-16 周：答辩准备
准备演示 Demo（录制视频或现场演示）。制作 PPT，完成毕业答辩。

六、 主要参考文献

[1] X. Wang, Y. Zhang, "A Survey on the Applications of Zero-Knowledge Proofs," arXiv preprint arXiv:2408.00243, 2024.

[2] "A Survey on the Applications of Zero-Knowledge Proofs," Aug 2024.

[3] "A Survey of Zero-Knowledge Proof Based Verifiable Machine Learning," arXiv preprint arXiv:2502.18535, 2025.

[4] "CrowdProve: Community Proving for ZK Rollups," arXiv preprint, 2025.

[5] ConsenSys, "Gnark: a fast zk-SNARK library in Go," GitHub Repository, 2024.

[6] "zkSaaS: Zero-Knowledge SNARKs as a Service," USENIX Security 2023.

[7] T. Liu, T. Xie, J. Zhang, et al., "Pianist: Scalable zkRollups via Fully Distributed Zero-Knowledge Proofs," IEEE S&P 2024.

[8] Y. Chen et al., "Advances in Zero-Knowledge Proofs: Bridging the Gap between Theory and Practice," UC Berkeley EECS Technical Report, 2024.

[9] Zellic, "Two Vulnerabilities in gnark's Groth16 Proofs," Zellic Research Blog, 2024.

[10] SotaZK, "PLONK: Permutations over Lagrange-Bases for Oecumenical Noninteractive Knowledge," SotaZK Labs docs, 2025.

[11] B. Chen et al., "HyperPlonk: Plonk with Linear-Time Prover and High-Degree Custom Gates," EUROCRYPT 2023.

[12] H. Wu, W. Zheng, A. Chiesa, et al., "DIZK: A Distributed Zero Knowledge Proof System," USENIX Security 2018.

[13] "Pianist: Scalable zkRollups via Fully Distributed Zero-Knowledge Proofs," ResearchGate.

[14] ConsenSys, "What's gnark," Documentation, 2025.

[15] "Scalable Collaborative zk-SNARK and Its Application to Fully Distributed Proof Delegation," USENIX Security 2025 Appendix.

[16] "Understanding PLONK," Medium, 2025.

[17] "Benchmark comparing zk-SNARK, zk-STARK, and Bulletproof in Real-World Scenarios," MDPI, 2024.

[18] ConsenSys, "gnark-crypto module," Go Packages, 2025.

[19] "Pianist: Scalable zkRollups via Fully Distributed Zero-Knowledge Proofs," Berkeley RDI.

[20] dreamATD, "pianist-gnark," GitHub Repository.

[21] A. Gabizon et al., "PLONK: Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge," Cryptology ePrint Archive, 2019.

[22] A. Ozdemir and D. Boneh, "Experimenting with Collaborative zk-SNARKs: Zero-Knowledge Proofs for Distributed Secrets," USENIX Security 2022.

<table>
<tr><td colspan="2">允许进入毕业设计（论文）下一阶段：是 ☐ 否 ☐</td><td rowspan="2">指导教师
签字</td><td rowspan="2"></td></tr>
<tr><td>日期</td><td>2025 年 12 月 27 日</td></tr>
</table>

注：可根据开题报告的长度加页